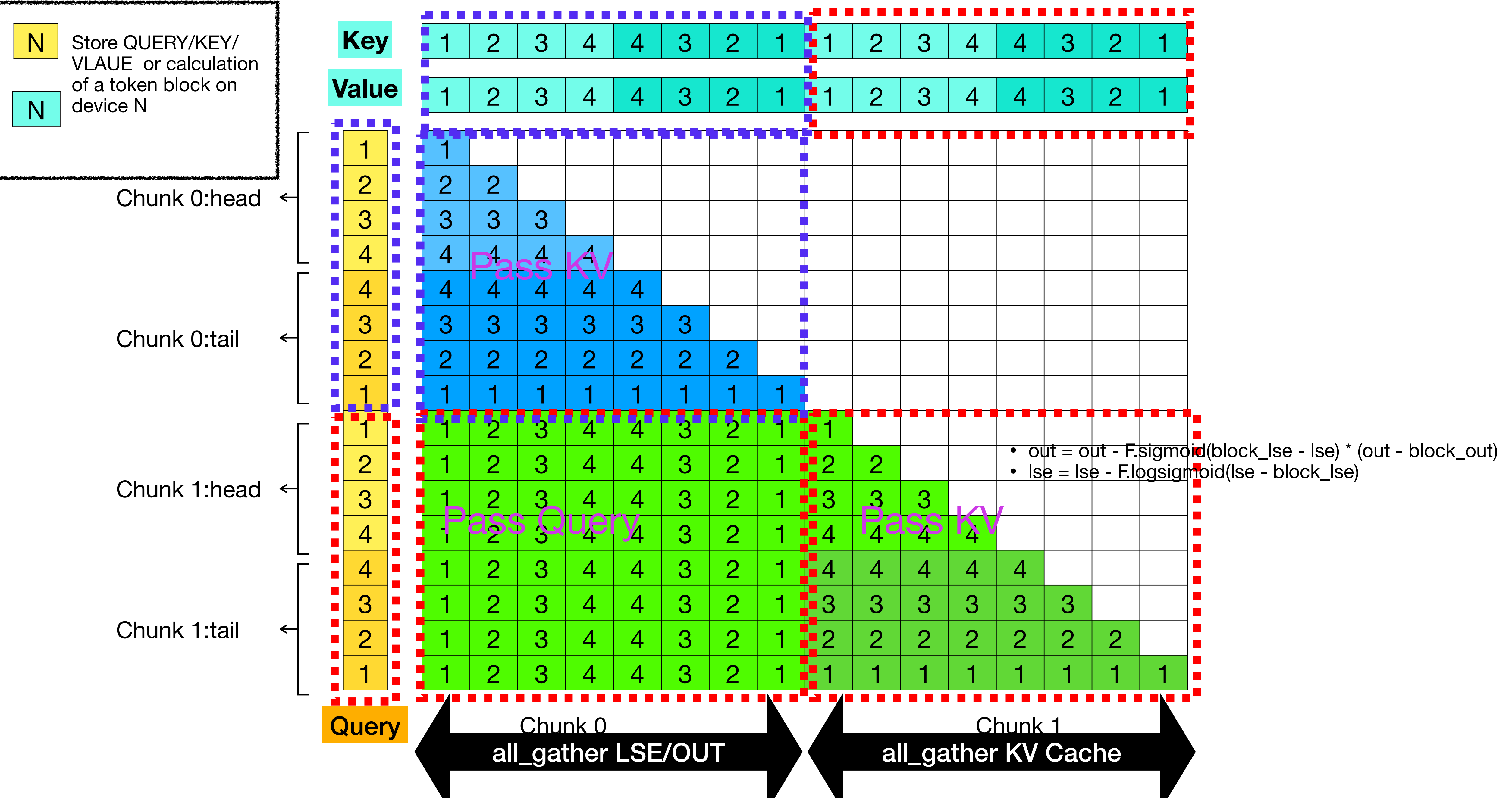


CP+chunked prefill 初步方案



Chunked prefill：传递kv 还是传递计算结果

1. 传递kv（需要管理kv，实现较麻烦）

- （方案1）沿用目前all_gather策略，按chunk执行prefill，每个chunk执行前先broadcast 历史 kv，再计算该chunk对应out和lse
- （方案2）Ring attention 方案，对于每个chunk按若干step计算，每个step 分别接收来自上一个device的kv，并将当前kv发送给下一个chunk

2. 传递计算结果（推荐，历史kv 保留在每个device上，传递每个chunk的输出跟lse）

- 历史 chunk：不用三角因果 mask，按 no_mask 计算本地 out、lse；
- 然后只把 out、lse 做 all_gather 并用稳定公式融合，更新累计的 out_acc、lse_acc。不传 KV。
- 本轮“当前分段”（CP 的 head/tail）：对“当前块”用三角因果 mask 计算；对“更早块”再做一次 no_mask 累加。
$$\text{out} = \text{out} - \text{F.sigmoid}(\text{block_lse} - \text{lse}) * (\text{out} - \text{block_out})$$
$$\text{lse} = \text{lse} - \text{F.logsigmoid}(\text{lse} - \text{block_lse})$$

相关修改

- mla_v1.py:_compute_prefill_context
 - 当前只考虑cp=1的情况，需增加cp>1, all gather 传递lse跟out 再聚合
- mal_v1.py:_forward_prefill
 - 合并历史chunk 的attention(来自_compute_prefill_context)与当前chunk的attention结果
- scheduler.py:schedule
 - 移除cp=1 限制

```
out_acc, lse_acc = out_head_tail, lse_head_tail # 已排序好的当前分段初值
for chunk in chunks:                          # 逐块计算chunked prefill的token
    out_loc, lse_loc = ring_mla_no_mask(q, KV_local(chunk))
    if cp_size > 1:
        out_loc, lse_loc = merge_cp_results(all_gather(out_loc), all_gather(lse_loc))
    out_acc, lse_acc = merge(out_acc, lse_acc, out_loc, lse_loc)
return out_acc, lse_acc
```

```
# 当前分段 (CP head/tail) :
out_cur, lse_cur = cp_head_tail(query_cur, KV_cur, mask_triu_then_nomask,
cp_allgather_KV)
# 历史 chunk 累加 :
out_all, lse_all = _compute_prefill_context(query_cur, kv_cache, meta, out_cur, lse_cur)

return out_all
```

时间估计

- 配置环境，跑通代码 (~2天)
- 初步实现 (2天)
- 测试+debug (5~8天)